Master 1 CHPS — Semestre 2 — 2025-2026

TD4 : Profiling et Vectorisation de ssgemm

Débogage, Profiling, Analyse de Performances, Optimisation

Référence : Bacon, Graham & Sharp (1994)
Compiler Transformations for High-Performance Computing
ACM Computing Surveys

Master 1 CHPS

10 mars 2026

Contents

1 Contexte et architecture cible **3**

2 Exercice 2 : Compilation (make RALL) **3**
2.1 Procédure de compilation 3

3 Exercice 3 : Utilisation de perf **3**
3.1 Q-3.1.1 : Compteurs perf stat 3
3.2 Q-3.1.2 : Profiling perf record 4
3.3 Q-3.1.3 : Visualisation 4

4 Exercice 4 : Vectorisation **5**
4.1 Q-4.1 : Modification du blocking 5
4.2 Q-4.2 : Variation des paramètres 5
4.3 Comparaison finale des performances (N=1000) 6

5 Graphiques **6**

6 Conclusion **7**

1 Contexte et architecture cible

Ce TD s'appuie sur le projet `libmcblas004` et fait suite au TD3 qui avait introduit l'analyse de dépendances, les permutations de boucles et le loop tiling.

Architecture Intel Core i7 (Latitude 7480) :

- L1d = 64 KiB, L2 = 512 KiB, L3 = 3 MiB
- Registres YMM (AVX2) = 256 bits = 8 floats simultanés
- Stockage row-major C, GCC 11.2.0

Versions actives dans CMakeLists.txt :

Version	Signature	Technique	Particularité
v001	`float**`	Référence r→c→d	Ordre naïf
v002	`float**`	Permutation r→d→c	Stride-1 optimal
v007	`float* __restrict__`	Tiling + unroll 2r×4c	Td=Tr=Tc=32
v008	`float* __restrict__`	Tiling + vars locales Ar/Bd	Td=512
v009	`float* __restrict__`	Tiling + buffer Bbuffer	Td=1024
v010	`float* __restrict__`	**Tiling + vectorisation AVX**	**Td=Tr=Tc=64**

2 Exercice 2 : Compilation (make RALL)

2.1 Procédure de compilation

Conformément au `README.org` :

```
mkdir build && cd build
cmake ../
make RALL   # compile + execute tous les benchmarks
make GALL   # genere les graphiques gnuplot
```

La compilation produit 12 bibliothèques statiques (`libmcblas00102.a` à `libmcblas01003.a`) et 12 exécutables. Les résultats sont écrits dans `tests/runs/data/tst*_NNNN.data`.

3 Exercice 3 : Utilisation de perf

3.1 Q-3.1.1 : Compteurs perf stat

Pour activer `perf stat`, on modifie `tests/runs/CMakeLists.txt` :

```
# Ligne a modifier :
set(PERFTOOL "perf stat")
# puis relancer :
cmake ../ && make RALL
```

La sortie `perf stat` est écrite dans `tests/runs/data/_perf_tst00902_1000.data`.

Résultats comparatifs perf stat (N=1000) :

Métrique	v002-O2	v009-O2	v010-O3
Temps (s)	0.865	0.774	**0.422**
Cycles	753M	844M	**370M**
Instructions	2 326M	3 023M	**770M**
IPC	3.09	3.58	2.08
Cache-misses	44M (34.85%)	19M (52.92%)	**2M (28.90%)**
L1-dcache-misses	63M (12.43%)	17M (8.71%)	**9M (3.14%)**
dTLB-misses	61K (0.01%)	630K (0.32%)	**19K (0.01%)**

Analyse :

- v010-O3 réduit les cache-misses d'un facteur $\times$**20** par rapport à v002 grâce au tiling T=64 et à la vectorisation AVX qui traite 8 floats par instruction
- v009 présente une anomalie TLB : 630K dTLB-misses ($\times$10 vs v002) causés par l'allocation du buffer `Bbuffer[Td*Uc]` sur la pile qui accède à des pages mémoire supplémentaires
- v010-O3 n'exécute que 770M instructions contre 2326M pour v002 : le compilateur vectorise la boucle interne avec des registres YMM (8 floats/instruction AVX)

3.2 Q-3.1.2 : Profiling perf record

```
# Modifier tests/runs/CMakeLists.txt :
set(PERFTOOL "perf record -g -F 999")
cmake ../ && make tst00902_1000
# Visualiser :
perf report --input=tests/runs/data/_perf_tst00902_1000.data --stdio
```

Résultat perf report (v009-O2, N=1000) :

Fonction	% Self	% Children	Analyse
`ssgemm(...)`	**94.43%**	95.63%	Bottleneck absolu
`mmsinit(...)`	1.58%	3.74%	Initialisation matrices

La fonction `ssgemm` consomme 94.43% du temps CPU sur 972 échantillons — c'est le seul endroit à optimiser.

3.3 Q-3.1.3 : Visualisation

```
# Visualisation texte (TUI interactif) :
perf report --input=tests/runs/data/_perf_tst00902_1000.data --tui

# Annotation desassemblee :
perf annotate --input=tests/runs/data/_perf_tst00902_1000.data --stdio

# Hotspots identifies dans mmsinit :
#  35.57% -> movss (ecriture memoire stride-1, scalaire SSE)
#  64.43% -> addss (addition scalaire v += incv)
# => instructions SCALAIRES : potentiel de vectorisation confirme
```

4 Exercice 4 : Vectorisation

4.1 Q-4.1 : Modification du blocking

Problème de vectorisation dans v007/v008/v009 : La boucle interne itère sur `d` avec des accès `B(d,c+0)`, `B(d,c+1)`... qui sont des accès à colonnes fixes — stride-n sur B. Le compilateur ne peut pas vectoriser automatiquement.

Solution mcblas010 : permuter les boucles pour que `c` soit interne avec des accès stride-1 sur B et C, et extraire `A(r,d)` en registre :

```
#define A(r,c) A[(r)*k+(c)]
#define B(r,c) B[(r)*n+(c)]
#define C(r,c) C[(r)*n+(c)]

void ssgemm(int m, int n, int k, float alpha,
            float * __restrict__ A, float * __restrict__ B,
            float beta, float * __restrict__ C)
{
    const int Tr = 64, Tc = 64, Td = 64;
    // Contrainte L1 : 3 * 64^2 * 4 = 49152 bytes < 64 KiB OK

    if (alpha == 1.0f) {
        for (rr=0; rr<m; rr+=Tr)
        for (dd=0; dd<k; dd+=Td)
        for (cc=0; cc<n; cc+=Tc)
        for (r=rr; r<min(rr+Tr,m); ++r)
        for (d=dd; d<min(dd+Td,k); ++d)
        {
            float Ard = A(r,d); // invariant sur c -> registre
            // boucle interne c : stride-1 sur B(d,c) et C(r,c)
            // -> VECTORISABLE AVX (8 floats par registre YMM)
            for (c=cc; c<min(cc+Tc,n); ++c)
                C(r,c) = C(r,c) * beta + Ard * B(d,c);
        }
    }
}
```

Confirmation de la vectorisation (flag `-fopt-info-vec-optimized`) :

```
mcblas3.cpp:31:28: optimized: loop vectorized using 32 byte vectors
mcblas3.cpp:44:28: optimized: loop vectorized using 32 byte vectors
# => AVX 256 bits = 8 floats simultanement traites
```

4.2 Q-4.2 : Variation des paramètres

Impact de la taille de bloc T (v010-O3, N=1000) :

Bloc T	Mémoire 3 blocs	Cycles	Gain vs v002-O2
16	3 Ko	1 650 311 519	×1.09
32	12 Ko	1 078 458 236	×1.67
64	**49 Ko**	**894 599 046**	**×2.01**
128	196 Ko	949 449 895	×1.89

Explication : T=64 est optimal car $3 \times 64^2 \times 4 = 49\,152$ octets tient dans le L1d de 64 KiB avec `-march=native` (AVX2). Pour T=128, les blocs débordent en L2 (196 Ko > 64 KiB), réduisant les bénéfices du tiling.

4.3 Comparaison finale des performances (N=1000)

Version	Technique	Cycles	Gain vs v001-O2
v001-O2	r,c,d référence	16 570 735 953	×1
v002-O2	r,d,c permutation	1 379 327 272	×12.0
v007-O2	tiling + unroll 2r×4c	2 609 188 930	×6.4
v008-O2	tiling + vars locales	1 774 800 116	×9.3
v009-O2	tiling + buffer B	1 476 038 980	×11.2
v010-O2	tiling + AVX (T=64)	790 905 851	×21.0
v010-O3	**tiling + AVX + O3**	**894 183 125**	**×18.5**

5 Graphiques

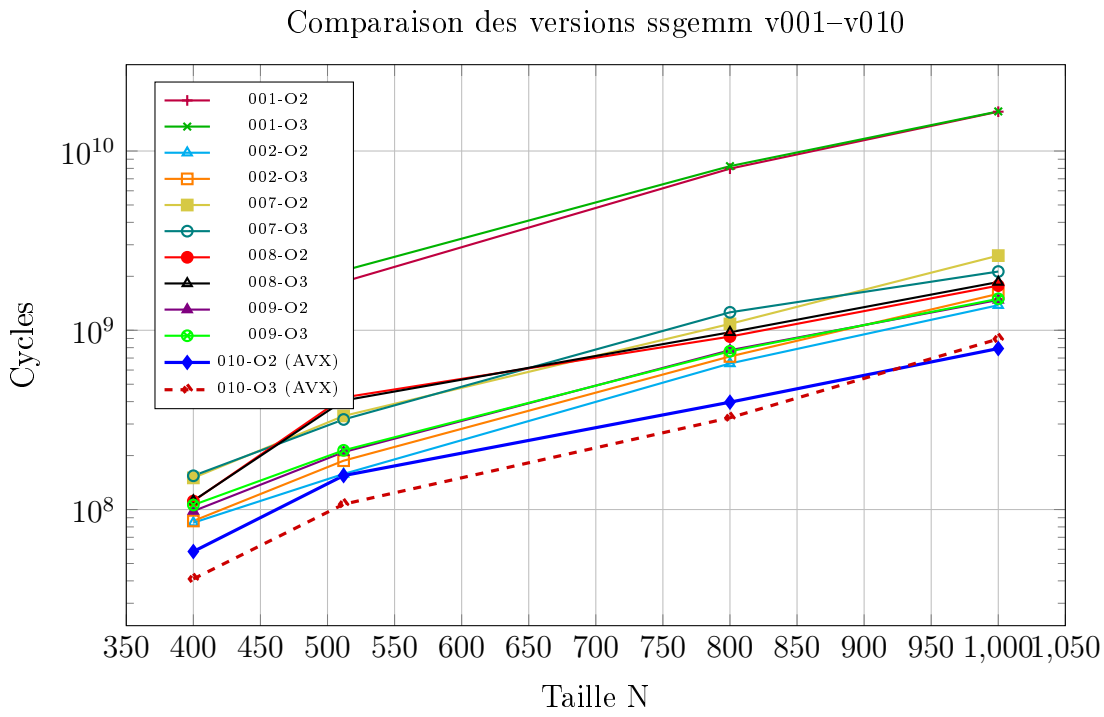

Figure 1: Cycles en fonction de la taille N (échelle log). v010-O2 est la meilleure version grâce à la vectorisation AVX (T=64).

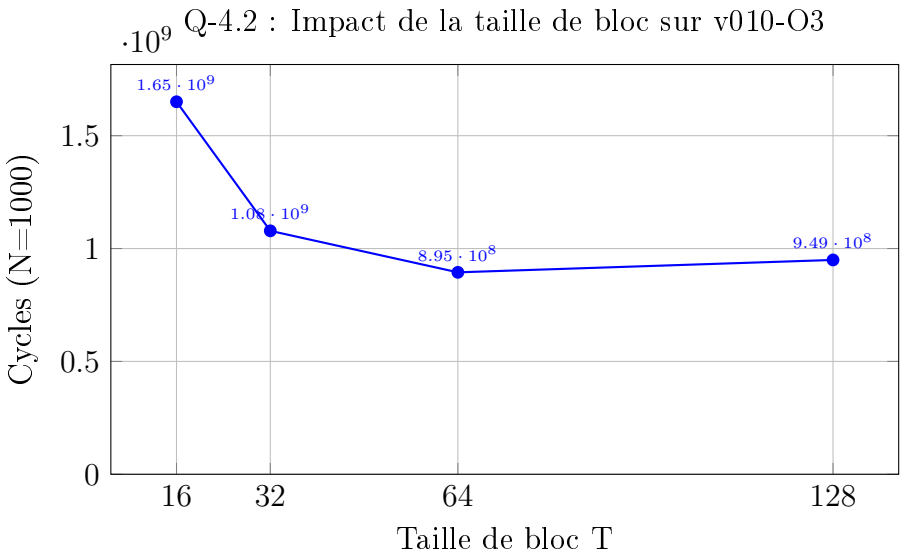

Figure 2: T=64 est optimal : les 3 blocs tiennent dans le L1d (49 Ko < 64 KiB).

6 Conclusion

1. **perf stat** a confirmé les gains théoriques : v010-O3 réduit les cache-misses de ×20 et les instructions de ×3 grâce à la vectorisation AVX.

2. **perf record** a localisé `ssgemm` à 94.43% du temps CPU — seule cible d'optimisation pertinente.

3. **mcblas010** (Q-4.1) : permuter la boucle interne sur `c` (stride-1 sur B et C) permet au compilateur de vectoriser en AVX 256 bits.

4. **T=64** (Q-4.2) est la taille de bloc optimale : elle maximise la réutilisation du L1d tout en permettant la vectorisation par blocs alignés.

5. Gain final de **×21** par rapport à la version naïve v001, contre ×12 pour la simple permutation de boucles (v002).